

CYBERMIND — Final Implementation Plan

ASUS ROG Strix G16 (Intel Core Ultra 9 275HX, RTX 5060 Laptop 8 GB) → NVIDIA Blackwell GB10 • SIH PS 26153 • NTRO

Grounded in **IconoclasticDev/SIH26 @ c2fe449**. Covers architecture additions, dataset acquisition, and edge deployment validation, distributed across phases rather than treated as separate tracks. Existing Phases 0–7 of the master implementation plan continue to apply around this.

Hardware, confirmed — ASUS ROG Strix G16. Intel Core Ultra 9 275HX (Arrow Lake, 8 P-cores + 16 E-cores, 24 cores / 24 threads, up to 5.4 GHz) paired with an NVIDIA GeForce RTX 5060 Laptop GPU (8 GB GDDR7, 128-bit bus, Blackwell architecture). System RAM varies by SKU (commonly 16–32 GB) — confirm with `free -h` before setting dataloader worker counts. GPU VRAM is fixed at 8 GB regardless of configuration, matching the existing `configs/laptop.yaml` ceiling of 6 GB usable. Run `nvidia-smi` once to confirm driver version and bf16 support before Phase 0.

Two genuine advantages of this exact machine. First, the RTX 5060 and the NVIDIA Blackwell GB10 share the same GPU architecture generation — precision behaviour (bf16, tensor-core code paths) that passes smoke tests here is far more likely to carry over unchanged to the GB10 than from an older-architecture laptop. Second, the 275HX’s 24 threads are real headroom for the dataloader — set `num_workers` to roughly 8–12 in the laptop-side smoke configs (unlike the GB10 config’s own `num_workers: 0`, which is a separate, already-flagged issue on that machine) so graph collation does not bottleneck the GPU during local smoke runs.

Sequencing rule, unchanged in spirit from the master plan: nothing in Phase 5 (GB10) starts until every phase below exits clean on the RTX 5060 laptop. The laptop is where correctness is proven; the GB10 is where scale is spent. Debugging a shape error on GB10 time is the single most avoidable waste in this project.

Phase summary

#	Hardware	Goal	Exit criterion
0	RTX 5060 laptop	Confirm hardware, capture baseline, kick off dataset acquisition	Baseline smoke metrics saved; knowledge bases downloaded; manual-portal requests submitted
1	RTX 5060 laptop (CPU+GPU)	Edge-conditioned attention, implemented	Forward+backward clean on both PyG and fallback paths
2	RTX 5060 laptop (CPU+GPU)	Kill-chain-constrained stage decoder, implemented	Decoder never emits an unmasked illegal transition
3	RTX 5060 laptop	Joint smoke integration, regression-safe	Both features toggle independently without breaking baseline
4	RTX 5060 laptop / docs	Config flags + ablation matrix + dataset validation gate	Four run configs exist, preflight clean, primary/validation datasets confirmed or escalated
5	GB10 (Blackwell)	Acquire primary corpus on host, then real training runs	Corpus validated strict-mode; at least baseline + combined checkpoint exist
6	GB10 / RTX 5060 laptop	Evaluation, calibration, ablation report, cross-dataset check	Metrics table complete; cross-dataset result present or explicitly marked pending access
7	Docs	Update submission package with evidence and dataset provenance	Novelty claim backed by the ablation; every dataset’s provenance documented
8	RTX 5060 laptop (CPU-only test)	Validate edge deployment: distill, export, verify CPU-only	Distilled ONNX model passes CPU-only latency/size test, confirming no-GPU deployability

Dataset acquisition reference (per the PS’s Dataset Link section)

These map directly to the datasets and knowledge bases the PS names. The repository already has scripted support for the low-effort ones (`scripts/download_public_sources.py`, `scripts/collect_public_corpus.py`, `scripts/prepare_cic2018_public.sh`) and a provenance manifest (`data/manifests/dataset_registry.csv`, `SOURCE_PROVENANCE.md`) for the rest. Nothing here is invented — this is what is already wired in `configs/sources.yaml`.

Source	Role	Acquisition	Needed by	Priority
CIC-IDS2018	Primary training corpus	Scripted — AWS S3, no-sign-request (<code>prepare_cic2018_public.sh</code>)	Phase 5	Blocking
MITRE ATT&CK (STIX)	Attack ontology / kill-chain grounding	Scripted — public STIX JSON (<code>download_public_sources.py --source attack_enterprise</code>)	Phase 0, informs Phase 2	Blocking, cheap

Source	Role	Acquisition	Needed by	Priority
CAPEC	Attack-pattern ontology	Scripted — public ZIP/XML snapshot	Phase 0	Blocking, cheap
NVD (CVE)	Vulnerability context	Scripted — public API snapshot (collect_knowledge.sh)	Phase 0	Blocking, cheap
CTU-13	Cross-dataset zero-shot validation	Manual — Stratosphere portal, no API	Phase 6.3	Blocking for that one check — start early, portal lead time is unpredictable
UNSW-NB15	Cross-dataset validation (optional second check)	Manual — UNSW research page	Phase 6.3 (optional)	Nice to have
CIC-IDS2017	Cross-dataset validation	Manual — UNB download form	Not on critical path	Nice to have
CICIoT2023	IoT/topology validation	Manual — UNB/cicresearch download	Not on critical path	Future work
LANL Authentication	Auxiliary identity/temporal context	Manual — LANL research portal	Not required	Future work
DARPA 1998/1999	Historical sequence validation	Manual — MIT Lincoln Lab pages	Not required	Future work
NCIIPC	CII context and contact, not a trainable dataset	Manual review only — do not scrape	Docs/context only	Reference only

Why this is distributed across phases rather than one dataset phase: the scriptable knowledge bases have no lead time and no dependency, so they run in Phase 0 alongside hardware confirmation. The primary corpus is 100+ GB and belongs on the GB10 host directly, not the laptop, so it is acquired in Phase 5, right before training. CTU-13 requires manual portal approval with unpredictable turnaround, so requesting access starts in Phase 0 even though the dataset itself is not used until Phase 6 — otherwise the wait becomes the critical path.

Phase 0 — Hardware confirmation and baseline capture

RTX 5060 LAPTOP

Before any new code is written, lock down what "working" currently looks like so a regression is visible immediately rather than discovered after both features are already tangled together.

0.1 Run `python scripts/inspect_env.py` and `nvidia-smi`; confirm driver version and bf16 support. GPU VRAM is fixed at 8 GB, so `configs/laptop.yaml` batch size stays as-is. Set `num_workers` in the smoke config to 8–12 to use the 275HX's 24 threads for dataloading rather than leaving it at the default.

0.2 Run the existing suite unmodified: `python -m pytest tests -q`, then `python scripts/phase01_smoke.py`, then `python scripts/train.py --config configs/smoke.yaml --device cpu --epochs 2`. Save the resulting `train_history.json` and `eval_test.json` as the baseline-before reference.

0.3 Kick off dataset acquisition in parallel with Phases 1–3 — none of it blocks code work, and some of it has lead time that will become the real bottleneck if left until later.

- Scriptable and cheap, run now: `python scripts/download_public_sources.py --source attack_enterprise --source capec_latest` for MITRE ATT&CK STIX and CAPEC, plus `scripts/collect_knowledge.sh` for an NVD snapshot. These ground the kill-chain transition structure used in Phase 2.

- Manual-portal datasets with unpredictable approval time — submit requests now: CTU-13 (Stratosphere portal, needed for Phase 6.3), and optionally UNSW-NB15 / CIC-IDS2017 / CICIoT2023. Do not wait until Phase 4 to start these; the portal turnaround, not the download itself, is what costs time.

- LANL and DARPA 1998/1999 are auxiliary/historical per `dataset_registry.csv` — request only if time allows, treat as future work otherwise.

Exit criterion: baseline smoke run completes with finite loss and is committed to `examples/smoke/` as the pre-change reference; knowledge-base downloads complete; manual-portal requests for CTU-13 are submitted. Every later phase compares against this baseline, not against memory of what used to work.

Phase 1 — Edge-conditioned graph attention

RTX 5060 LAPTOP

Wires the already-computed `edge_attr` (protocol, port, bytes, duration, IAT statistics) into the graph attention mechanism, which currently only sees node-level aggregates.

1.1 Confirm `edge_attr` shape and dtype at the point `graph_builder.py` emits it — no changes needed here, this is a read-only check.

1.2 `graph_encoder.py` — extend `GATv2GraphEncoder.__init__` to pass `edge_dim=edge_attr_dim` into both `GATv2Conv` layers, and change `forward(self, x, edge_index)` to `forward(self, x, edge_index, edge_attr=None)`, passing it through to both convolutions when present.

1.3 Extend `DenseGraphAttention` (the no-PyG fallback) so its score function becomes a function of source, destination and edge features, not source/destination alone — otherwise the fallback path silently keeps the current blind spot even after the fix.

1.4 `world_model.py` — update `encode_state`, `forward` and `forward_batch` to thread `state.edge_attr` through to the graph encoder call.

1.5 Add a feature flag `model.use_edge_features: true|false` so the old behaviour remains reachable for the ablation in Phase 6 — do not delete the no-edge-features path.

1.6 Shape/gradient smoke test: synthetic 2-graph, 3-node batch, both `use_edge_features: true` and `false`, both the PyG path and the forced-fallback path (temporarily uninstall or monkeypatch `HAS_PYG` to confirm). Assert finite loss and non-zero gradient norm on the new edge-projection weights specifically, not just on the total loss.

Precision/FPR rationale: attention can now down-weight a large benign transfer on a common port differently from a similarly large transfer on an unusual port — the exact disambiguation that global feature normalisation (existing Phase 0.1 of the master plan) cannot provide on its own.

Exit criterion: all four combinations (edge features on/off × PyG/fallback) run one forward+backward pass on CPU and on the RTX 5060 laptop GPU with finite loss and non-zero relevant gradients. Do not proceed to Phase 2 until this is green.

Phase 2 — Kill-chain-constrained stage decoding

RTX 5060 LAPTOP

Replaces independent per-window stage classification with a decoder that enforces plausible ATT&CK kill-chain ordering, so a single noisy window cannot flip the predicted stage backward and forward without cause.

2.1 Define the allowed-transition structure. In `knowledge/stage_mapping.yaml`, add a `transition_matrix` section: `forward-or-stay` transitions through `Benign` → `Reconnaissance` → `Initial Access` → `Lateral Movement` → `Command & Control` → `Exfiltration` are allowed; an explicit "campaign reset" transition back to `Benign` or `Reconnaissance` is allowed; all other backward jumps are masked to `-inf`. `Unknown/Ambiguous` is exempted from the constraint so genuinely unclear windows are not forced into a false ordering.

2.2 Add a small linear-chain CRF layer (a learned transition-score matrix plus the existing per-window `StageHead` logits as emissions) in a new `src/cybermind/models/stage_decoder.py`. Mask disallowed transitions per 2.1 before the forward/Viterbi computation. Keep `StageHead` itself unmodified — it still produces emissions; the CRF sits on top.

2.3 `losses.py` — add a `crf_stage_loss` (CRF negative log-likelihood over the true stage sequence) alongside the existing `stage_loss`. Wire it into `train.py`'s `batch_loss` behind a `loss.use_crf_stage` flag so the plain-softmax path stays available for the ablation.

2.4 Replace `argmax` stage decoding with constrained Viterbi decoding at inference time, in `eval.py`, `app.py`, and `world_model.forecast` where stage sequences are produced from rollouts.

2.5 New metric: *illegal-transition rate* — fraction of consecutive predicted-stage pairs in a sequence that violate 2.1's ordering, excluding declared resets. Add to `src/cybermind/evaluation/metrics.py`. This is the concrete, measurable signal that this phase is doing what it claims.

2.6 Shape/decode smoke test: synthetic per-window logits fed through the CRF decoder; assert (a) finite CRF loss, (b) decoded sequences from a small adversarial synthetic case never contain an unmasked illegal transition, (c) gradients reach the new transition-matrix parameters.

Precision/FPR rationale: a window independently misclassified as `Command & Control` between two `Reconnaissance` windows currently stands as-is; the CRF structurally suppresses it unless the emission evidence is strong enough to justify a reset, which is a tighter and more principled constraint than the existing post-hoc persistence rule (master plan 3.3).

Exit criterion: CRF forward/backward and constrained decode both pass on CPU and RTX 5060 laptop GPU with a synthetic batch, illegal-transition rate is computably zero on the adversarial synthetic case, and the plain-softmax path still works with the flag off.

Phase 3 — Joint smoke integration and regression check

RTX 5060 LAPTOP

Confirms the two features do not interact badly with each other or with the existing pipeline before any GB10 time is spent.

3.1 Run `python -m pytest tests -q` and `python scripts/phase01_smoke.py` with both `use_edge_features` and `use_crf_stage` set to every combination of `true/false` (four runs).

3.2 Confirm the both-false run reproduces the Phase 0 baseline metrics within floating-point tolerance — this is the regression guarantee, not a nice-to-have.

3.3 Confirm the both-true run trains for two epochs on the smoke config without NaNs, without exceeding the RTX 5060 laptop's confirmed VRAM, and with the illegal-transition-rate metric reporting a sane, non-degenerate number (not identically 0% or 100%, which would indicate the mask is either not applied or is collapsing all predictions to one stage).

Exit criterion: all four combinations run cleanly on synthetic/subsampled data on the laptop. This is the actual gate before Phase 4 begins.

Phase 4 — Config and ablation matrix preparation

RTX 5060 LAPTOP / docs

4.1 Add both feature flags to `configs/gb10_full.yaml`, and create three sibling configs — `gb10_edge_only.yaml`, `gb10_crf_only.yaml`, `gb10_baseline.yaml` — that differ only in these two flags, so the four training runs in Phase 5 are directly comparable.

4.2 Run `scripts/gpu_preflight.py` against all four configs on the RTX 5060 laptop with a tiny synthetic batch to catch config-level shape errors before GB10 time is spent on them.

4.3 Extend `eval.py` to log the illegal-transition rate and, if `use_edge_features` is on, the mean attention weight assigned to edges above a chosen bytes/port suspicion heuristic — a cheap qualitative check that the new signal is

actually being used, not just present.

4.4 Confirm the primary corpus situation before finalising configs: `gb10_full.yaml` sets `require_packet_features: true`, so `prepare_data.py --strict` will hard-fail if only flow-only CIC-IDS2018 CSVs are available with no PCAP-derived packet attributes. Verify PCAP availability now — this determines whether Phase 5 can start on schedule.

4.5 Check status of the Phase 0.3 CTU-13 portal request. If access has not been granted, flag the Phase 6.3 cross-dataset check as at-risk now rather than discovering it during evaluation.

Exit criterion: four config files exist, all pass preflight on the laptop, the evaluation pipeline can report the two new diagnostic metrics end to end on a smoke checkpoint, and the primary-corpus packet-feature question has a definite answer.

Phase 5 — GB10 training runs

NVIDIA BLACKWELL
GB10

This is the first phase in this document that spends GB10 time. Everything above must be green first.

5.0 Acquire the primary corpus directly on the GB10 host, not the laptop — at 100+ GB it belongs on the machine with the storage and bandwidth to use it. Install the AWS CLI and run `scripts/prepare_cic2018_public.sh` (or `download_public_sources.py --source cic2018_s3`), then `build_corpus.py → validate_dataset.py --strict → prepare_data.py --strict` per the master plan's existing Phase 2 sequence. Do not proceed to training until strict validation passes.

5.1 Run `gb10_baseline.yaml` first, always — this is both the master plan's own Phase 2 run and the reference point every ablation number in Phase 6 is measured against.

5.2 If the time budget allows only one more run, run the combined config (`gb10_full.yaml` with both flags on) next — it is the highest expected-value run for both the precision story and the novelty story.

5.3 If time remains, run `gb10_edge_only.yaml` and `gb10_crf_only.yaml` to isolate each feature's individual contribution — this is what turns the novelty claim into an ablation table rather than an assertion.

5.4 Log all four loss components separately per run, per the master plan's existing Phase 2.3 practice, plus the CRF loss and illegal-transition rate where applicable.

Exit criterion: at minimum, baseline and combined checkpoints exist. Isolated-feature checkpoints are valuable but not blocking for the submission if the window runs out.

Phase 6 — Evaluation, calibration, and the ablation report

GB10 / RTX 5060
LAPTOP

6.1 Apply the master plan's threshold-calibration sweep (3.2) identically to every variant that was trained — same target FPR, same validation split — so precision/recall comparisons across variants are on equal footing rather than at each variant's own arbitrary cutoff.

6.2 Build the metrics table: precision, recall, F1, FPR, and illegal-transition rate for baseline vs. edge-only vs. CRF-only vs. combined, at the matched calibrated threshold.

6.3 Cross-dataset zero-shot check (master plan 3.4) on CTU-13, run only on the best-performing variant if time is short — this evaluation can run on the RTX 5060 laptop since it is inference-only. If the Phase 0.3 portal request has not been granted by this point, report the check as *pending data access* in the submission rather than omitting it silently — an honestly labelled gap is better than a missing section with no explanation.

Exit criterion: one table exists showing whether edge features, the CRF decoder, or both measurably reduce FPR relative to baseline at a matched operating point. If a variant does not help, say so in the report — an honest null result is still evidence, and this is exactly what an ablation is for.

Phase 7 — Submission documentation update

DOCS ONLY

7.1 Update the architecture document with the two new components and one diagram showing where edge features enter attention and where the CRF sits over the stage head.

7.2 Rewrite the novelty section to cite the Phase 6 ablation numbers directly, replacing any architectural claim that is not backed by a measured comparison.

7.3 If time allows, surface the illegal-transition-rate metric and per-edge attention weight in the demo UI next to the existing risk timeline — this doubles as a live demonstration of the ablation story during the video.

7.4 Document dataset provenance in the README/architecture doc: source URL, acquisition method, and SHA256/manifest reference for every dataset actually used, drawn from `data/manifests/SOURCE_PROVENANCE.md` and `dataset_registry.csv`. The PS asks for reproducible setup instructions; this is what makes the data side of that claim checkable rather than asserted.

Exit criterion: every novelty and precision claim in the final submission traces to a specific row in the Phase 6 table, and every dataset used has a traceable, cited source.

Phase 8 — Edge deployment validation (CPU-only, no GPU required)

RTX 5060 LAPTOP
(CPU-ONLY TEST)

Confirms the trained model can actually run on an average machine with no dedicated GPU — the demo/deployment target implied by the PS's offline, CII-suitable interface requirement. This is validation of existing tooling, not new development.

8.1 Run `scripts/distill_student.py` against the best GB10 checkpoint from Phase 5/6, producing a small student model sized to `configs/laptop.yaml` (hidden=128, 4 heads) rather than the full GB10 training width (hidden=512, 8 heads).

8.2 Run `scripts/export_teacher_onnx.py` against the distilled student to produce the ONNX artifact.

8.3 Run `tests/test_onnx_cpu.py` on the RTX 5060 laptop with CUDA disabled (or on any spare CPU-only machine) to confirm it stays under the enforced 100 MB size and 50 ms per-inference budget on `CPUExecutionProvider` alone — this is the concrete, existing test that proves no-GPU deployability, not an assumption.

8.4 Smoke-test `app.py` itself with CUDA hidden from the process (`CUDA_VISIBLE_DEVICES=""`) to confirm the demo interface loads and runs end to end on CPU alone, matching what a judge or an air-gapped CII deployment would actually see.

Honest caveat: the 50 ms/100 MB budget is a single-inference benchmark, not a throughput test at real network scale (many hosts and flows at once). It confirms the model *can* run without a GPU; it does not yet confirm sustained throughput under load — that remains open per the practicability gap already noted in the master plan.

Exit criterion: distilled ONNX model passes the CPU-only latency/size test, and the demo app runs end to end with no GPU visible to the process.

If GB10 time runs out after the baseline run only: ship the baseline submission per the master plan unchanged, and present Phases 1–4 of this document as verified, tested, ready-to-train code with a stated expected effect — an honestly labelled "implemented and unit-tested, not yet trained at scale" status is far more credible than an untested architectural claim dressed up as a result.